

İçindekiler

Yönetici Özeti	5
Mevcut Mimari	6
1.1 IPC Mimarisi (stdin/stdout JSON-line)	6
1.2 Dizin Yapısı	6
1.3 Mimari Diyagram (Mevcut)	7
Mevcut Özellikler	9
2.1 Dava Türü Tespiti (10 tür)	9
2.2 Dilekçe Üretimi (12 tür)	9
2.3 Kalite Kontrol (17 boyut)	9
2.4 Halüsinasyon Kontrolü	9
2.5 İhtihat Doğrulama	10
2.6 Hukuk Kaynağı Araştırması (6 kaynak)	10
2.7 PDF Üretimi (browser subprocess)	10
2.8 AI Ajan Entegrasyonları (4 ajan)	10
Semantic Search Durumu	11
3.1 Modern Arama Bileşenleri — Mevcut Durum	11
3.2 Mevcut Arama Stratejisi: Uzak Portala Delege	11
3.3 Memory Mekanizması (Yerel Cache)	12
3.4 Web Platformu İçin Gereken Yeni Arama Katmanı	12
AI Sistemi Durumu	13
4.1 Kullanılan Modeller	13
4.2 Üç Inference Yolu	13

4.3 Agentic Multi-Turn (MAX_TURNS=8)	13
4.4 Prompt Sistemi	13
4.5 AI Karar Verme Noktaları	14
4.6 Web Platformu İçin AI Provider Abstraction	14
MCP Durumu	15
5.1 MCP Server Mimarisi	15
5.2 Kayıtlı 21 MCP Aracı	15
5.3 Connect Konfigürasyonları	16
5.4 Web'e Taşınabilirlik	16
Veri Modeli	17
6.1 Mevcut SQLite Şema	17
6.2 Mevcut Şemanın Sorunları	17
6.3 PostgreSQL Migration Stratejisi	18
Web'e Taşınabilir Kodlar	19
7.1 Doğrudan Taşınabilir Modüller	19
7.2 Adapter İle Taşınabilir Modüller	19
Yeniden Yazılması Gereken Kodlar	21
8.1 engine.py — 7.857 Satırlık Tek Dosya	21
8.2 LLM Hattı — Provider Abstraction	21
8.3 Memory Katmanı — JSON Path → PostgreSQL/Redis	21
8.4 HTML/PDF/DOCX Render — Yeni Katman	21
8.5 IPC/WebView → REST/SSE	21
8.6 license_gate.py — Kaldırılmalı	22
8.7 Search-Motor Entegrasyonu — Yok → Eklenecek	22

8.8 Semantic Search — Sıfırdan Yazılacak	22
Riskler ve Risk Matrisi	23
9.1 Risk Önceliklendirme	24
Önerilen Next.js Mimarisi	25
10.1 Mimari Diyagram (Önerilen)	25
10.2 Katman Sorumlulukları	26
10.3 İletişim Akışı	27
PostgreSQL + pgvector Mimarisi	28
11.1 Auth & Tenant Tabloları	28
11.2 Case & Document Tabloları	29
11.3 Legal Source Tabloları (9M+ karar)	30
11.4 Petition & Research Tabloları	31
11.5 pgvector Index Stratejisi	33
API Taslağı	34
12.1 Auth Endpoints	34
12.2 Legal Search Endpoints	34
12.3 Petition & Research Endpoints	35
12.4 Document, Chat & Admin Endpoints	35
Migration Planı (19 Faz)	37
13.1 Kritik Yol (Critical Path)	38
Tahmini Modül Listesi	39
14.1 Frontend Modülleri (app/)	39
14.2 Backend Modülleri	39
14.3 Shared Modüller	40

İlk Sprint Görevleri	41
15.1 Sprint 2 Önizleme	41
15.2 Sprint 3 Önizleme	41
Sonuç ve Sonraki Adımlar	42
Hemen Başlanacak İlk 3 Adım	42
Korunması Gereken Değerler (Hatırlatma)	42

Yönetici Özeti

BetterSaul projesi, Türkiye hukuk kaynakları üzerinde çalışan yerel (desktop) bir AI destekli hukuk araştırma ve dilekçe üretim platformudur. Mevcut sistem; **15.318 satır Python kodu**, 7.857 satırlık tek bir *engine.py* dosyası, 21 MCP aracı, 6 farklı hukuk portalı entegrasyonu (Yargıtay, Danıştay, Emsal, AYM, Resmî Gazete, mevzuat.gov.tr), tamamen yerel Qwen 2.5/3 modelleri ve Edge WebView2 tabanlı bir masaüstü arayüzünden oluşmaktadır.

Bu rapor, mevcut mimarinin derinlemesine analizini yapmakta ve Next.js + TypeScript + PostgreSQL + pgvector tabanlı modern, ölçeklenebilir bir web platformuna dönüşüm planını sunmaktadır. Analiz sonucunda tespit edilen **kritik bulgular**: (1) Mevcut sistemde **semantic search tamamen yok** — embedding, vektör veritabanı, BM25, reranking veya query expansion mekanizmalarının hiçbirisi mevcut değil; arama tamamen Yargıtay/Danıştay gibi uzak portallara HTTP POST delege edilmiştir. (2) AI inference tamamen yerel Qwen modellerine bağlıdır — hiçbir sağlayıcı çeşitliliği (OpenAI, Gemini, Anthropic) yoktur. (3) archive.db SQLite veritabanı yalnızca 2 tablodan oluşmakta, çok-kullanıcılı mimari, audit log, soft-delete veya versiyonlama desteklenmemektedir. (4) engine.py'nin 7.857 satırlık tek dosya yapısı bakım ve test açısından yüksek risk taşımaktadır.

Buna karşılık, mevcut sistem **korunması gereken değerli varlıklara** da sahiptir: 17 boyutta puanlama yapan deterministik kalite kontrol motoru (regex-tabanlı, LLM-as-judge bağımlılığı yok), halüsinasyon kontrolü (TCKN/tanık/tutar uydurma tespiti), 10 dava türü otomatik tespiti, 12 dilekçe şablonu, prompt sızıntısı tespiti ve içtihat doğrulama. Bu bileşenlerin tamamı saf Python olup, doğrudan FastAPI endpoint'lerine sarılarak web platformunda yeniden kullanılabilir.

Önerilen migration planı **19 fazdan** oluşmaktadır ve toplam 5-7 ay sürmesi öngörülmektedir. İlk 3 faz (Kod Analizi, DB Şema, Python Engine Servisleştirme) tamamlanmıştır veya bu raporla başlatılmıştır. Kritik dönüşüm alanları: (a) PostgreSQL + pgvector ile gerçek **hybrid search** (semantic + keyword + reranking), (b) AI **provider abstraction** katmanı, (c) **multi-tenant** auth ve RBAC, (d) **versiyonlu mevzuat** ve dilekçe sürüm geçmişi, (e) **research trace** ile AI şeffaflığı, (f) **document upload** ile kullanıcının kendi belgeleri üzerinde semantic search.

BÖLÜM 1

Mevcut Mimari

BetterSaul, Windows platformuna özel olarak geliştirilmiş, üç katmanlı bir masaüstü uygulamasıdır. Üst katmanda .NET (C#) ile yazılmış bir WinForms ana uygulama ve gömülü Microsoft Edge WebView2 bulunur. Orta katmanda, kullanıcıdan bağımsız bir süreç olarak çalışan Python 3.12 engine yer alır. Alt katmanda ise SQLite tabanlı yerel arşiv ve 6 hukuk portalına yapılan HTTP çağrılarını bulunur. Tüm bu katmanlar, **stdin/stdout JSON-line IPC** ile birbirleriyle haberleşir — HTTP/socket kullanılmaz.

1.1 IPC Mimarisi (stdin/stdout JSON-line)

engine.py'nin **main()** fonksiyonu (satır 7816) bir stdin döngüsüdür. WinForms uygulamasından gelen her satır JSON komut olarak parse edilir, dispatch edilir ve sonuç **emit()** fonksiyonu (satır 87-90) ile tekrar JSON olarak stdout'a yazılır. Bu mimari, bir Unix pipe'ına benzer şekilde çalışır:

```
# engine.py satır 87-90
def emit(event: str, text: str = "") -> None:
    with _emit_lock:
        sys.stdout.write(json.dumps({"event": event, "text": text}, ensure_ascii=False) + "\n")
        sys.stdout.flush()
```

WinForms tarafı bu stdout satırlarını okur, JSON parse eder ve WebView2'ye *postMessage* ile iletir. **app.js** (73KB, 1790 satır IIFE) bu mesajları dinler ve DOM günceller. Bu mimarinin web'e taşınması için IPC katmanının tamamen kaldırılıp REST/SSE/WebSocket ile değiştirilmesi gerekir.

1.2 Dizin Yapısı

Dizin	İçerik	Boyut / Satır
app/engine/	Ana Python motoru (engine.py + prompts + legal_tracks + petition_packs)	8.664 satır
app/engine/bettersaul_mcp/	MCP paketi: server, sources, anayasa, mevzuat, petition_tools, quality, templates, motor/	5.131 satır
app/ui/	Vanilla JS SPA: index.html + app.js + app.css + net.js	~110KB / 1790 satır JS
archive.db	SQLite: 2 tablo (chats, petitions)	24KB · 0 satır veri
runtime/python/	Bundled Python 3.12 Windows AMD64 + uvicorn + mcp SDK	~150MB
webview/EBWebView/	Edge WebView2 user data dizini (cache, cookies, GPU)	~80MB
certs/	Self-signed TLS: mcp-cert.pem, mcp-key.pem	CN=BetterSaul MCP

Dizin	İçerik	Boyut / Satır
connect/	5 AI ajan MCP config: Claude, ChatGPT, Cursor, Gemini (JSON+TOML)	5 dosya
logs/mcp.log	MCP server runtime log	852 byte · 28 satır

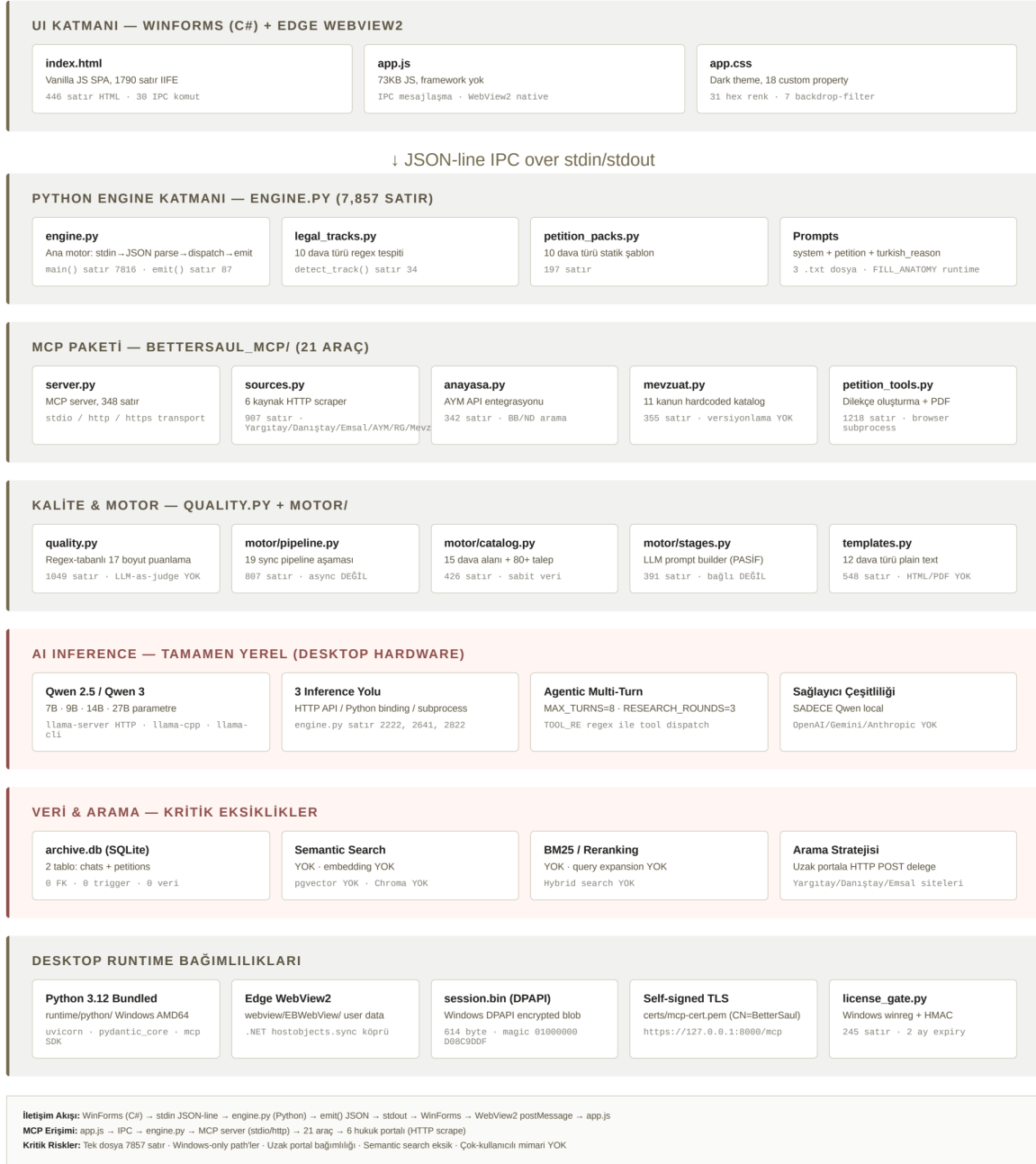
Tablo 1.1 — BetterSaul proje dizin yapısı

1.3 Mimari Diyagram (Mevcut)

Aşağıdaki diyagram, mevcut BetterSaul mimarisinin 7 katmanını özetlemektedir: UI katmanı (WebView2), Python engine, MCP paketi, kalite kontrol motoru, AI inference, veri ve arama katmanı, desktop runtime bağımlılıkları. Kırmızı ile işaretli katmanlar web'e taşırken kritik risk taşıyan bölümleri gösterir.

MEVCUT BETTERSaul MİMARİSİ

Desktop (Windows) · WebView2 IPC · Local Qwen · MCP over stdio/HTTP



Şekil 1.1 — Mevcut BetterSaul mimari diyagramı

BÖLÜM 2

Mevcut Özellikler

BetterSaul'un mevcut sürümü, hukuk araştırması ve dilekçe üretimi için geniş bir özellik seti sunar. Aşağıda her özellik grubu, dosya:satır referanslarıyla birlikte listelenmiştir. Bu özelliklerin korunması ve web platformunda yeniden sunulması, migration'ın temel hedefidir.

2.1 Dava Türü Tespiti (10 tür)

legal_tracks.py (satır 34, *detect_track()*) regex tabanlı bir dava türü sınıflandırıcıdır. 10 dava alanı tanır: iş, boşanma, kira, tüketici, alacak, tazminat, icra, idare, vergi, tapu/miras. Her track için **PROMPT_RULES** (satır 405-420) usul kurallarını barındırır; örneğin işçilikte 7036 m.3/5, 4857 m.32/34/41/17/18/20/21, 1475 m.14 ve HMK m.107 atıfları modele gömülür. Bu kurallar, dilekçe üretimi sırasında AI'ya hangi mevzuatı kullanması gerektiğini söyleyen bir usul rehberi görevi görür.

2.2 Dilekçe Üretimi (12 tür)

templates.py (satır 8-197, *KINDS* sözlüğü) 12 dava türü için şablon tanımlar: idare, boşanma, iş, icra, tüketici, kira, tazminat, ceza, alacak, diğer, tapu, miras. Her tür; mahkeme başlığı, iptal flag'i (yalnızca idare), sections (koşullu bloklar), anayasa maddeleri ve ilgili kanun maddelerini içerir. **petition_packs.py** (197 satır) ise 10 dava türü için statik hazır şablonlar sunar — bu, AI'nin sıfırdan yazmasını gerektirmeyen, doldurulan form'a göre özelleştirilen hazır metinlerdir.

Dilekçe üretim akışı 4 aşamalıdır: **_pipe_extract** (satır 7658) → **_pipe_generate** (satır 7681) → **_pipe_check** (satır 7711) → **_pipe_repair** (satır 7743). İlk aşamada kullanıcı formundan yapılandırılmış kart çıkarılır, ikinci aşamada LLM dilekçe metni üretir, üçüncü aşamada 17-boyut kalite kontrol çalışır, dördüncü aşamada ise skoru düşükse LLM'den düzeltme istenir.

2.3 Kalite Kontrol (17 boyut)

quality.py (1.049 satır) + **motor/pipeline.py** (807 satır) birlikte 17 boyutta puanlama yapan bir kalite kontrol sistemi oluşturur. Boyutlar: taraf tutarlılığı, tutar çelişkileri, mevzuat grafiği, talep-KONU-SONUÇ üçlüsü, dayanak-talep uyumu, iddia-delil eşleşmesi, kronoloji, yetki/görev, usul ön şartları, çelişkiler, halüsinasyon, içtihat doğrulama, içtihat-kalem uyumu, vakıa-hukuk-delil-talep zinciri, KVKK/kişisel veri, prompt sızıntısı, zorunlu alan eksikleri. Genel puan 0-100 arası, KRİTİK bulgu varsa ≤ 79 .

2.4 Halüsinasyon Kontrolü

Halüsinasyon kontrolü tamamen **kaynak-karşılaştırma** yöntemiyle yapılır — LLM-as-judge kullanılmaz. İki yerde uygulanır: **_invent_findings** (quality.py satır 744-787) ve **_hallucination** (pipeline.py satır 487-509). TCKN (11 haneli) regex ile yakalanır, form'da olup olmadığı kontrol edilir; tanık adları regex ile çıkarılır, fold edilmiş form içeriğinde aranır; ISO tarihler yakalanır; **PROMPT_LEAK** tablosu (catalog.py satır

421-426: "prompt", "AI", "wikipedia", "Vekile Not", "[DOĞRULANACAK", "Yargı MCP" vb.) ile sistem mesajı sızıntısı tespit edilir.

2.5 İctihat Doğrulama

`_cite_findings` (quality.py satır 790) dışarıdan verilen *memory* sözlüğündeki **cites** ve **aym** listeleriyle dilekçe metnindeki tüm `E.\s*\d{4}/\d+,?\s*K\s*\d{4}/\d+` veya `B.\s*No\s*[: ]?\s*\d{4}/\d+` numaralarını karşılaştırır. Memory'de kayıtlı olmayan numara → YÜKSEK "doğrulanmamış içtihat" uyarısı. Bu mekanizma, AI'nin uydurma karar numarası üretmesini engeller. **Ancak** bu mekanizma **semantic içtihat araması yapmaz** — yalnızca bire-bir esas-no eşleşmesi yapar.

2.6 Hukuk Kaynağı Araştırması (6 kaynak)

Kaynak	URL / Endpoint	Erişim	Dosya
Yargıtay	karararama.yargitay.gov.tr	HTTP POST + HTML scrape	sources.py satır 600
Danıştay	danistaydergiler.adalet.gov.tr	HTTP POST + HTML scrape	sources.py
Emsal (UYAP)	emsal.uyap.gov.tr	HTTP POST + HTML scrape	sources.py satır 691
Anayasa Mahkemesi	anayasa.gov.tr/api/core/public/search	JSON API	anayasa.py satır 147
Resmî Gazete	resmigazete.gov.tr	HTTP scrape	sources.py satır 828
Mevzuat	mevzuat.gov.tr	HTTP scrape (11 hardcoded kanun)	mevzuat.py

Tablo 2.1 — BetterSaul tarafından desteklenen hukuk kaynakları

2.7 PDF Üretimi (browser subprocess)

`petition_tools.py` satır 1046-1096 (`_print_pdf`) PDF üretimi için sistemdeki Edge veya Chrome browser'ı `--headless` modda subprocess olarak çağırır. Bu yöntem web ortamında çalışmaz — bunun yerine WeasyPrint veya Playwright + `page.pdf()` kullanılmalıdır. **DOCX çıktısı yoktur**; HTML ve plain text çıktısı mevcuttur.

2.8 AI Ajan Entegrasyonları (4 ajan)

`connect/` klasörü 5 farklı AI ajan için MCP config dosyası içerir: Claude Desktop, ChatGPT Desktop (JSON + TOML), Cursor, Gemini. Hepsi aynı `run_mcp.py --stdio` script'ine işaret eder. HTTP endpoint olarak `https://127.0.0.1:8000/mcp` (self-signed TLS) kullanılır. **chatgpt.toml** dosyasında `default_tools_approval_mode = "auto"` ayarı vardır — yani ChatGPT kullanıcı onayı olmadan MCP araçlarını çağırabilir. Bu, web platformunda güvenlik açısından dikkatle ele alınmalıdır.

BÖLÜM 3

Semantic Search Durumu

Bu bölüm raporun **en kritik** bölümüdür. Kullanıcının beklentileri ve modern hukuk araştırma platformlarının gereksinimleri doğrultusunda, mevcut BetterSaul sisteminde semantic search altyapısının **hiç olmadığı** tespit edilmiştir. Aşağıdaki tablo, modern bir hukuk araştırma platformunda bulunması beklenen tüm arama bileşenlerini ve BetterSaul'deki durumlarını listeler.

3.1 Modern Arama Bileşenleri — Mevcut Durum

Bileşen	BetterSaul Durumu	Açıklama
Keyword Search (yerel)	YOK	Yerel FTS5/Whoosh yok; phrase uzak portala delege edilir
Semantic Search	YOK	Hiçbir vektör/semantic eşleştirme yok
Embedding	YOK	OpenAI/Cohere/BGE/e5/mxbai çağrısı yok
Vector Database	YOK	pgvector/Chroma/Faiss/Pinecone/Weaviate/Qdrant yok
BM25	YOK	rank_bm25/bm25_okapi yok, TF-IDF hesabı yok
Reranking	YOK	Cross-encoder/LLM-rerank/Cohere-rerank yok
Metadata Filtering (yerel)	YOK	Yerel post-filter yok; court_types uzak portal filtresi
Query Expansion	YOK	Ne LLM-rewrite ne thesaurus; sadece stop-word temizliği
Hybrid Search	YOK	Sparse+dense birleştirme yok
Belge Chunking	YOK	Kararlar/metinler chunklanmıyor
Caching	YOK	Her seferinde uzak portal çağrılır

Tablo 3.1 — Modern arama bileşenleri ve BetterSaul durumu

3.2 Mevcut Arama Stratejisi: Uzak Portala Delege

BetterSaul'de yerel bir arama motoru yoktur. `sources.py::search_corpus()` (satır 600) kullanıcıdan gelen phrase'i doğrudan Yargıtay portalına POST'lar; cevap JSON/HTML olarak parse edilir ve sonuçlar olduğu sırayla döndürülür. `search_corpus_deep()` (satır 619) sayfa sayfa dolaşarak daha fazla sonuç toplar; `search_bedesten()` (satır 661) Bedesten (Yargıtay+Danıştay+Emsal) pazarlama adıyla aynı işi yapar. `anayasa.py::search_api()` (satır 147) AYM'nin resmi JSON API'sine POST yapar.

Bu yaklaşımın **4 kritik dezavantajı** vardır: (1) **Portal bağımlılığı** — Yargıtay/Danıştay siteleri erişime kapanırsa veya yapı değişirse sistem çalışmaz. (2) **Hız** — her sorgu 3-15 saniye sürer (ağ + parse + rate-limit cooldown). (3) **Esneklik yok** — "güvenlik soruşturması nedeniyle atanamama" sorgusu yalnızca aynı

kelimeleri içeren kararları getirir; "arşiv araştırması", "memuriyet", "ölçülülük" gibi anlamsal bağlantılı kavramlar bulunamaz. (4) **Sıralama opak** — uzak portalın kendi sıralaması kullanılır, yerel rerank yok.

3.3 Memory Mekanizması (Yerel Cache)

Sistemde yerel bir cache mekanizması olarak **case_memory.json** dosyası kullanılır (*quality.load_memory* satır 926). Bu dosya macOS'ta `~/Library/Application Support/BetterSaul/`, Windows'ta `%LOCALAPPDATA%/BetterSaul/` altında saklanır. Memory, daha önceki oturumda çekilmiş içtihat künyelerini ve AYM karar numaralarını tutar. **quality.review** fonksiyonu bu memory'yi alır ve dilekçedeki tüm atıfları memory'deki listede arar. Bu bir cache'dir, semantic bir index değildir.

3.4 Web Platformu İçin Gereken Yeni Arama Katmanı

Mevcut sisteme semantic search eklemek, sıfırdan yeni bir katman yazmayı gerektirir. Önerilen pipeline: **Document** → cleaning → normalization → chunking (ör. 512 token, 50 overlap) → metadata extraction (court, chamber, decision_no, date, type, source) → embedding (multilingual-e5-large 1024d veya OpenAI text-embedding-3-large 1536d) → PostgreSQL + pgvector (HNSW index). Sorgu zamanı: **kullanıcı sorgusu** → query expansion (LLM ile sinonim/kavram genişletme) → parallel (a) semantic search pgvector'dan, (b) keyword search PostgreSQL FTS'den, (c) metadata filter → sonuçları birleştir → cross-encoder rerank (top-50 → top-10) → kullanıcıya döndür.

BÖLÜM 4

AI Sistemi Durumu

BetterSaul'ün AI katmanı tamamen **yerel (local) inference**'a dayanır. Bulut AI sağlayıcıları (OpenAI, Google Gemini, Anthropic) hiç kullanılmaz. Bu tasarım kararı, gizlilik ve veri主权 açısından güçlü olmakla birlikte, donanım bağımlılığı ve ölçeklenebilirlik açısından zayıftır. Web platformuna taşırken **provider abstraction** katmanı eklenmesi kritik bir gereksinimdir.

4.1 Kullanılan Modeller

Sistem **Qwen 2.5** serisinden 7B, 9B ve 14B parametrelili modeller ile **Qwen 3** serisinden 14B ve 27B parametrelili modelleri destekler. Model seçimi *runtime/config*'ten yapılır. Bu modeller Türkçe için optimize edilmiş olup, hukuk alanında makul performans gösterir. Ancak GPT-4o veya Claude 3.5 Sonnet gibi frontier modellere kıyasla karmaşık akıl yürütme ve uzun bağlam görevlerinde geridedirler.

4.2 Üç Inference Yolu

engine.py üç farklı yoldan LLM çağırısı yapabilir. Bu, donanım ve işletim sistemi uyumluluğu için bir yedekleme mekanizması sağlar:

Yöntem	Mekanizma	Konum
llama-server HTTP	C++ binary ayrı process, HTTP API (OpenAI uyumlu)	engine.py satır 2641
llama-cpp-python	Python binding, doğrudan process içinde GGUF yükle	engine.py satır 2222
llama-cli subprocess	CLI çağırısı, stdin/stdout pipe	engine.py satır 2822

Tablo 4.1 — Üç inference yolu

4.3 Agentic Multi-Turn (MAX_TURNS=8)

engine.py **agentic multi-turn** bir mimari kullanır. **MAX_TURNS=8** sabiti (engine.py başı) bir kullanıcı sorgusunda AI'nin en fazla 8 tur araç çağırabileceğini belirtir. **RESEARCH_ROUNDS=3** ise araştırma turlarının sayısıdır — her turda AI sırayla içtihat, mevzuat, AYM ve Resmî Gazete kaynaklarını tarar. Tool dispatch mekanizması **TOOL_RE** regex'i ile yapılır (satır 2219): AI'nin cevabındaki `{"tool": "search_corpus", "args": {...}}` gibi JSON blokları regex ile yakalanır ve ilgili fonksiyon çağırılır.

4.4 Prompt Sistemi

Prompt sistemi 4 bileşenden runtime'da birleştirilir: (1) **system_prompt.txt** (22 satır) — temel sistem rolü; (2) **petition_prompt.txt** (112 satır, A-M bölümlü) — dilekçe yazma disiplini; (3) **turkish_reason.txt** (13 satır) — Türkçe akıl yürütme kuralları; (4) **FILL_ANATOMY** — runtime'da form verisinden oluşturulan dava bilgileri. Bu birleştirme engine.py içinde yapılır ve LLM'e tek bir system mesajı olarak gönderilir.

4.5 AI Karar Verme Noktaları

- **Dava türü tespiti:** `legal_tracks.detect_track()` regex ile (satır 34) — AI bağımsız değil
- **Araştırma sorgusu üretimi:** LLM, kullanıcı olayından 3-5 arama phrase'i üretir
- **Kaynak seçimi:** `legal_tracks.PROMPT_RULES` (satır 405-420) regex-tabanlı zorunlu atıflar
- **Araştırma turları:** AI sırayla içtihat (3 tur) → mevzuat → AYM (idare türünde 1) → Resmî Gazete
- **Sonuç filtreleme:** `_is_portal_junk()` (satır 1182) ile portal junk sonuçları elenir
- **Citation gerçeklik:** `_cite_from_mapping()` (satır 1053) ile yalnızca gerçek künyeler kullanılır
- **Dilekçe üretimi:** 4 aşamalı pipeline (extract → generate → check → repair)
- **Kalite onayı:** 17-boyut puanlama, 80 puan altı repair tetikler

4.6 Web Platformu İçin AI Provider Abstraction

Mevcut sistemdeki Qwen-only yaklaşımı web platformunda sürdürmek pratik değildir. Çünkü: (a) Her kullanıcının donanımı LLM çalıştırmak için yeterli değildir; (b) Server-side GPU maliyeti yüksektir; (c) Farklı görevler için farklı modeller optimumdur (ör. araştırma için GPT-4o, dilekçe için Claude). Önerilen **provider abstraction** katmanı şu sağlayıcıları desteklemelidir: OpenAI, Google Gemini, Anthropic, OpenRouter, Ollama (legacy Qwen uyumluluğu için). Sağlayıcı seçimi `AI_PROVIDER` env değişkeni ile yapılır. **API anahtarları asla frontend'e gönderilmez** — yalnızca backend'te tutulur.

BÖLÜM 5

MCP Durumu

Model Context Protocol (MCP), BetterSaul'un dış AI ajanlarına (Claude Desktop, ChatGPT Desktop, Cursor, Gemini) araç sağlamak için kullandığı standarttır. **server.py** (348 satır) MCP SDK üzerine kurulu tek bir *server* nesnesi tanımlar ve 21 aracı *@server.tool* dekoratörü ile kaydeder. Sınıf yapısı yoktur — tüm araçlar fonksiyon olarak tanımlanır.

5.1 MCP Server Mimarisi

Server üç transport modu destekler: (1) **stdio** — *run_stdio()* satır 307, stdin/stdout üzerinden; (2) **http** — satır 313-348, *stateless_http=True, json_response=True*; (3) **https** — *uvicorn+SSL* ile satır 321-339. Endpoint sabit olarak */mcp* kullanılır. **mcp.log** dosyasından anlaşılan: her HTTP request sonrası session kapanıyor (stateless mod), 21 "Terminating session" mesajı görülüyor.

5.2 Kayıtlı 21 MCP Aracı

Araç	İşlev	Web API Karşılığı
search_bedesten	Yargıtay+Danıştay+Emsal araması	POST /api/legal/search/bedesten
search_corpus	Yargıtay içtihat araması	POST /api/legal/search/yargitay
search_corpus_deep	Sayfa sayfa derin arama	POST /api/legal/search/deep
search_emsal	Emsal karar arama	POST /api/legal/search/emsal
search_mevzuat	Mevzuat arama (11 kanun)	POST /api/legal/search/mevzuat
search_anayasa	AYM kararı arama (BB/ND)	POST /api/legal/search/aym
search_resmi_gazete	Resmî Gazete arama	POST /api/legal/search/resmi-gazete
get_bedesten_document	Belge detayı ID ile	GET /api/legal/document/{id}
write_court_petition	Dilekçe oluştur	POST /api/petitions/generate
review_petition	Dilekçe kalite kontrol	POST /api/petitions/review
save_petition_pdf	PDF üret (browser subprocess)	POST /api/petitions/{id}/export-pdf
save_petition_history	Dilekçe arşivle (SQLite)	POST /api/petitions/{id}/save
ping / echo	Sağlık kontrolü	GET /api/health
(diğer 8 araç)	Yardımcı araçlar (license, version vb.)	Çeşitli endpoint'ler

Tablo 5.1 — 21 MCP aracı ve REST API karşılıkları

5.3 Connect Konfigürasyonları

connect/ klasörü 5 AI ajan için MCP entegrasyon config dosyası içerir. Hepsi aynı *run_mcp.py --stdio* script'ine işaret eder. **chatgpt.toml** dosyasındaki *default_tools_approval_mode = "auto"* ayarı, ChatGPT'nin kullanıcıdan onay almadan MCP araçlarını çağırabileceği anlamına gelir — bu güvenlik açısından risklidir ve web platformunda dikkatle ele alınmalıdır.

5.4 Web'e Taşınabilirlik

21 MCP aracından **18'i doğrudan REST API'ye çevrilebilir** — saf Python, OS bağımsız, hiçbir desktop bağımlılığı yok. Yalnızca **3 araç desktop bağımlıdır**: (1) *write_court_petition* — SQLite archive.db'ye yazar; (2) *save_petition_history* — aynı şekilde SQLite; (3) *save_petition_pdf* — Edge/Chrome browser'ı subprocess olarak çağırır ve Windows *os.startfile* kullanır. Bu üç araç, PostgreSQL + WeasyPrint kullanılarak yeniden yazılmalıdır.

MCP sisteminin tamamen kaldırılması zorunlu değildir. İleride dış AI ajanlarının (Claude Desktop vb.) BetterSaul web platformunu kullanabilmesi için **ayrı bir MCP endpoint** korunabilir. Bu endpoint, REST API'lerin üzerine bir MCP adapter katmanı olarak implemente edilir. Böylece hem insan kullanıcılar REST API'yi kullanır hem de AI ajanları MCP üzerinden erişir.

BÖLÜM 6

Veri Modeli

BetterSaul'ün veri katmanı, **archive.db** adlı 24KB'lık bir SQLite veritabanından ibarettir. Bu veritabanı yalnızca 2 tablo içerir: *chats* ve *petitions*. Toplam 0 satır veri, 0 foreign key, 0 trigger, 0 view barındırır. WAL mode aktif, page_size 4096, user_version 0 (migration yok). İki basit index vardır: *idx_chats_created* ve *idx_petitions_created* (created_at DESC).

6.1 Mevcut SQLite Şema

```
-- Tablo 1: chats (7 kolon)
CREATE TABLE chats (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT,
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  messages    TEXT,          -- JSON array
  track       TEXT,          -- dava türü
  summary     TEXT
);
CREATE INDEX idx_chats_created ON chats(created_at DESC);

-- Tablo 2: petitions (23 kolon - AŞIRI DENORMALİZE)
CREATE TABLE petitions (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  petition_type TEXT,
  petitioner_name TEXT,
  petitioner_tckn TEXT,
  petitioner_address TEXT,
  defendant_name TEXT,
  defendant_address TEXT,
  court       TEXT,
  case_no     TEXT,
  subject     TEXT,
  facts       TEXT,
  legal_basis TEXT,
  requests    TEXT,
  evidence    TEXT,
  body_text   TEXT,
  quality_score INTEGER,
  quality_report TEXT,      -- JSON
  status      TEXT,
  pdf_path    TEXT,
  notes       TEXT,
  metadata    TEXT          -- JSON grab-bag
);
CREATE INDEX idx_petitions_created ON petitions(created_at DESC);
```

6.2 Mevcut Şemanın Sorunları

- **Çok-kullanıcılı mimari yok:** user_id kolonu yok, tüm dilekçeler tek kullanıcıya ait varsayılıyor
- **Organizasyon/tenant yok:** multi-tenant SaaS için altyapı yok
- **Versiyonlama yok:** dilekçe güncellendiğinde eski sürüm kaybolur (no petition_versions tablosu)
- **Audit log yok:** kim ne zaman ne değiştirdi takip edilmiyor
- **Soft-delete yok:** silinen kayıt geri alınamaz
- **FK yok :** case-petition-document ilişkisi yok
- **Denormalize:** petitions tablosu 23 kolon, lawyer/araflar/deliller gömülü
- **Vector/embedding yok:** pgvector veya benzeri hiç yok
- **Legal source verisi yok:** içtihat/mevzuat/AYM kararları saklanmıyor, her seferinde uzaktan çekiliyor
- **Subscription/plan yok:** SaaS iş modeli için altyapı yok

6.3 PostgreSQL Migration Stratejisi

Mevcut 2 tablo yaklaşımı, web platformunda **18+ tabloya** bölünmelidir. *petitions* tablosu en az 4 tabloya ayrılır: *petitions* (ana kayıt), *lawyer_profiles* (vekıl bilgileri), *petition_claims* (talep kalemleri), *petition_research_items* (AI araştırma sonuçları). Ayrıca eksik tablolar eklenir: *users*, *organizations*, *subscriptions*, *api_keys*, *cases*, *case_parties*, *documents*, *document_chunks*, *document_embeddings*, *legal_sources*, *legal_decisions*, *statutes*, *statute_articles*, *citations*, *research_sessions*, *petition_versions*, *petition_reviews*, *ai_runs*, *audit_logs*. Detaylı şema Bölüm 11'de verilmiştir.

BÖLÜM 7

Web'e Taşınabilir Kodlar

BetterSaul kod tabanının yaklaşık **%60-65'i doğrudan web platformuna taşınabilir**. Bu bölüm, hangi modüllerin/fonksiyonların nasıl taşınacağını detaylandırır. Taşınabilirlik değerlendirmesi 4 kategoriye ayrılır: (a) doğrudan taşınabilir (saf Python, OS bağımsız), (b) adapter ile taşınabilir (küçük değişiklik), (c) yeniden tasarlanması gereken (büyük değişiklik), (d) silinecek (desktop-only).

7.1 Doğrudan Taşınabilir Modüller

Modül / Fonksiyon	Satır	Taşınma Stratejisi
<code>quality.review()</code>	945	Saf Python, sync. FastAPI endpoint'e sar: <i>await run_in_threadpool(quality.review, ...)</i>
<code>quality.preflight()</code>	1026	Aynı — preflight endpoint
<code>quality.format_report()</code>	988	Saf Python, hiç I/O yok
<code>quality.extract_model()</code>	323	Form → yapılandırılmış kart çıkarımı
<code>motor.pipeline.analyze()</code>	681	19 sync aşama, threadpool'da çalıştır
<code>motor.pipeline.classify()</code>	132	Naif keyword sınıflandırıcı, doğrudan taşınabilir
<code>motor.catalog.*</code>	1-426	Sabit veri modülü, JSON olarak DB'ye yüklenebilir
<code>templates.assemble()</code>	438	Plain text dilekçe üretimi, saf Python
<code>templates.skeleton()</code>	527	Dilekçe iskelet, saf Python
<code>templates.outline()</code>	256	Outline üretimi, saf Python
<code>templates.guide()</code>	484	AI usul rehberi metni, saf Python
<code>motor.stages.*</code>	1-391	LLM prompt builder (pasif), bağlanarak kullanılabilir
<code>legal_tracks.detect_track()</code>	34	Regex dava türü tespiti, saf Python
<code>petition_packs.*</code>	1-197	Statik şablon verisi, JSON'a taşınabilir
18/21 MCP aracı	—	Doğrudan FastAPI router'lara çevrilebilir (Bölüm 12)

Tablo 7.1 — Doğrudan taşınabilir modüller (~%60-65)

7.2 Adapter İle Taşınabilir Modüller

Bu modüller küçük adaptasyonlarla web'e taşınabilir:

- **sources.py (907 satır)**: sync *htpx.Client* → async *htpx.AsyncClient*; global *_LIMITED_UNTIL* → Redis; desktop UA korunabilir
- **anayasa.py (342 satır)**: aynı async dönüşümü, *_KUNYE* regex korunur
- **mevzuat.py (355 satır)**: async dönüşüm, hardcoded 11 kanun katalogu veritabanına taşınabilir
- **tls.py (48 satır)**: *truststore.SSLContext* → *certifi* (sunucu tarafında OS truststore gerekmez)
- **ping.py (40 satır)**: doğrudan taşınabilir, health endpoint

BÖLÜM 8

Yeniden Yazılması Gereken Kodlar

Bu bölüm, mevcut BetterSaul kod tabanında **yeniden tasarlanması veya baştan yazılması** gereken bileşenleri listeler. Bu bileşenler ya desktop-only bağımlılıklar içeriyor ya da web ölçeğinde çalışmayacak mimari kararlar barındırıyor.

8.1 engine.py — 7.857 Satırlık Tek Dosya

Bu en büyük taşıma riskidir. 7.857 satırlık tek bir Python dosyası, modüler olmayan yapıda, ~200 fonksiyon içerir. Web platformunda bu dosya **modüllere bölünmelidir**: *engine/main.py* (orchestrator), *engine/llm.py* (AI inference + provider abstraction), *engine/research.py* (araştırma pipeline), *engine/petition.py* (dilekçe pipeline), *engine/cleaner.py* (prompt sızıntısı temizleme), *engine/citation.py* (citation gerçeklik kontrolü). Her modül ~500-1500 satır olmalıdır.

8.2 LLM Hattı — Provider Abstraction

Mevcut sistem yalnızca Qwen 2.5/3 modellerini (llama-server/llama-cpp/llama-cli) destekler. Web platformunda **provider abstraction** katmanı yazılmalıdır. Bu katman, tüm sağlayıcıları (OpenAI, Gemini, Anthropic, OpenRouter, Ollama) tek bir arayüz altında birleştirir. **engine.py** satır **2222, 2641, 2822**'deki 3 inference yolu bu abstraction ile değiştirilir. Sağlayıcı seçimi *AI_PROVIDER* env değişkeni ile runtime'da yapılır.

8.3 Memory Katmanı — JSON Path → PostgreSQL/Redis

quality_load_memory (satır 926-942) desktop path'lerden *case_memory.json* yükler. Bu dosya macOS'ta *~/Library/Application Support/BetterSaul/*, Windows'ta *%LOCALAPPDATA%/BetterSaul/* altındadır. Web platformunda bu veri **PostgreSQL JSONB kolonunda veya Redis'te** saklanmalıdır. Çok-kullanıcılı olduğunda session/tenant ayrımı gerekir.

8.4 HTML/PDF/DOCX Render — Yeni Katman

templates.assemble() yalnızca plain text üretir — HTML, PDF, DOCX renderlayıcı **yok**. Web platformunda bu 3 renderer eklenmelidir: (a) **Jinja2** — HTML şablonlar; (b) **WeasyPrint** — HTML→PDF (mevcut *_print_pdf* browser subprocess yerine); (c) **python-docx** — DOCX üretimi (yoktan var edilecek). Bu üç bileşen *petition-renderer* servisinde toplanır.

8.5 IPC/WebView → REST/SSE

Mevcut UI ile backend arasındaki **stdin/stdout JSON-line IPC** tamamen kaldırılmalıdır. Yerine: (a) **REST API** — Next.js API routes üzerinden; (b) **SSE (Server-Sent Events)** — AI chat streaming için (chat-delta, petition-done event'leri); (c) **WebSocket** — opsiyonel, real-time collaborative editing için. *app.js*'deki 30 IPC

komutu → REST endpoint'lerine, 30 event → SSE event'lerine çevrilir.

8.6 license_gate.py — Kaldırılmalı

license_gate.py (245 satır) Windows *winreg*, hardcoded build tarihi (2026-09-20), 2 ay expiry, HMAC imza ve anti-rollback (internet zamanı) kullanır. Web platformunda bu mekanizma **tamamen kaldırılmalıdır**. Yerine SaaS subscription modeli (Stripe ile entegre) + JWT tabanlı auth gelir. Aktivasyon mekanizması yoktur — zaten desktop'ta da tam çalışmıyordur.

8.7 Search-Motor Entegrasyonu — Yok → Eklenecek

Mevcut durumda **motor tarama yapmaz**. *motor.pipeline.analyze* ve *motor.stages* modüllerinin hiçbir fonksiyonu *sources.py*'i çağırır. *quality._cite_findings* bir *memory* argümanı alır ama bu memory dışarıdan verilir; motor kendisi tarama yapmaz. Kullanıcı manuel arar, sonuçları *case_memory.json*'a kaydeder, sonra *quality.review* memory'yi okur. Web platformunda bu akış **otomatikleştirilmelidir**: form doldurulur → motor otomatik içtihat taraması yapar → sonuçları review'a besler.

8.8 Semantic Search — Sıfırdan Yazılacak

Mevcut sistemde semantic search **tamamen yok** (Bölüm 3). Web platformunda bu **tamamen yeni bir katman** olarak eklenmelidir. Bileşenler: (a) embedding pipeline (multilingual-e5-large veya OpenAI text-embedding-3-large); (b) PostgreSQL + pgvector (HNSW index); (c) hybrid search orchestrator (semantic + keyword + metadata); (d) cross-encoder reranker (bge-reranker-tr); (e) query expansion (LLM tabanlı sinonim/kavram genişletme). Mevcut kodda bu katmanın hiçbir iskeleti yoktur.

BÖLÜM 9

Riskler ve Risk Matrisi

Aşağıdaki tablo, migration sürecinde karşılaşılabilecek riskleri **etki (Impact)** ve **olasılık (Probability)** 维度 değerinde matris olarak sunar. Her risk için azaltma stratejisi (mitigation) önerilmiştir. Risk kodu: **K**=Kritik, **Y**=Yüksek, **O**=Orta, **D**=Düşük.

#	Risk	Etki	Olasılık	Azaltma Stratejisi
R 1	engine.py tek dosya 7.857 satır — bakım/test riski	K	Y	İlk sprint'te modüllere böl: main/llm/research/petition/cleaner/citation
R 2	Semantic search eksikliği — kullanıcı beklentisi karşılanmıyor	K	Y	pgvector + e5-large + cross-encoder rerank ile sıfırdan ekle
R 3	Uzak portal scraping bağımlılığı (Yargıtay/Danıştay)	Y	Y	Belge cache + Redis rate-limit + proxy rotation + monitoring
R 4	Local LLM donanım bağımlılığı — server GPU maliyeti	Y	Y	Provider abstraction ile OpenAI/Gemini fallback; Ollama opsiyonel
R 5	Windows path hardcoded (LOCALAPPDATA, winreg)	O	Y	Path yerine environment-driven config + PostgreSQL storage
R 6	license_gate Windows-only (winreg)	D	K	Tamamen kaldır, SaaS subscription (Stripe) ile değiştir
R 7	HTML/PDF rendering browser subprocess (Edge/Chrome)	O	Y	WeasyPrint + python-docx ile değiştir
R 8	Tek-kullanıcılı archive.db — multi-tenant yok	Y	Y	PostgreSQL 18+ tablo + org_id partitioning + RBAC
R 9	Prompt sızıntısı regex ile yakalanma — yeni pattern'ler kaçabilir	O	O	LLM-as-judge opsiyonel ikinci katman (motor.stages.check_messages)
R 10	9M+ belge import sırasında embed maliyeti	Y	Y	Async job queue + batch processing + increment indexing
R 11	ChatGPT default_tools_approval_mode="auto" — güvenlik	Y	O	Web platformunda her tool çağrısı için explicit user consent
R 12	TOOL_RE regex ile tool dispatch — JSON parse hatası	O	O	OpenAI function calling veya structured output kullan
R 13	Türkçe hukuk terminolojisi için embedding kalitesi	Y	O	multilingual-e5-large + Türkçe fine-tune veya BGE-m3
R 14	Uzak portal 429 rate-limit cooldown 50 sn — UX yavaş	O	Y	Redis cooldown + proxy pool + cache sonuçlar

Tablo 9.1 — Risk matrisi (Etki/Olasılık) ve azaltma stratejileri

9.1 Risk Önceliklendirme

Kritik (K) etkiye sahip riskler: R1 (engine.py modülerleştirme), R2 (semantic search eksikliği). Bunlar migration'ın başarısı için **ilk 3 sprintte** çözülmelidir. **Yüksek etki + Yüksek olasılık** riskleri (R3, R4, R8, R10) orta vadede (sprint 4-10) ele alınmalıdır. **Düşük etki** riskleri (R6) erken kaldırılmalı; **Orta etki** riskleri (R5, R7, R9, R12) kademeli olarak iyileştirilmelidir.

BÖLÜM 10

Önerilen Next.js Mimarisi

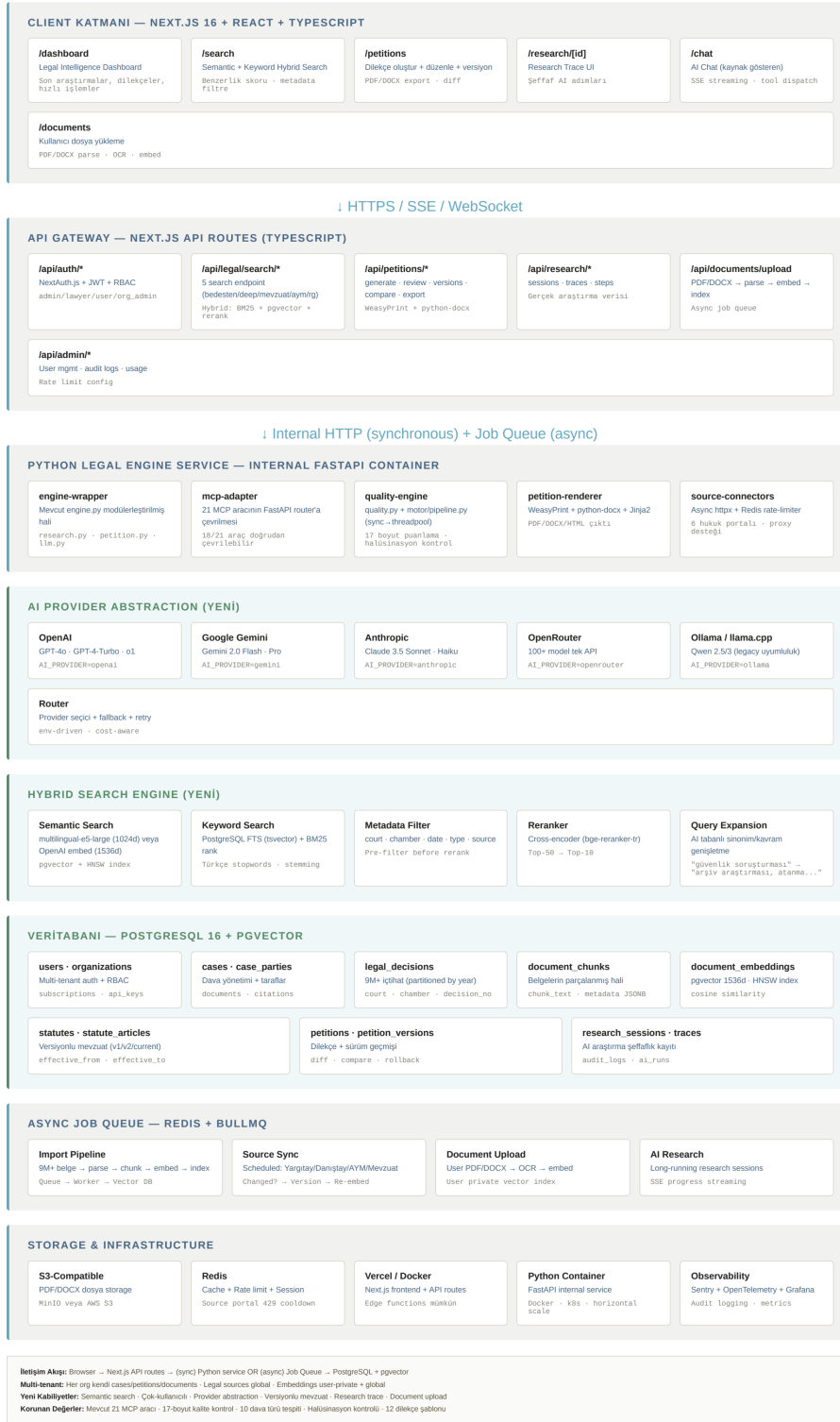
Önerilen web platformu mimarisi, 8 katmandan oluşur: (1) Client katmanı (Next.js), (2) API Gateway, (3) Python Legal Engine Service, (4) AI Provider Abstraction, (5) Hybrid Search Engine, (6) PostgreSQL + pgvector, (7) Async Job Queue, (8) Storage & Infrastructure. Her katmanın belirli sorumlulukları vardır ve birbirinden bağımsız olarak ölçeklendirilebilir.

10.1 Mimari Diyagram (Önerilen)

Aşağıdaki diyagram, önerilen mimarinin tüm katmanlarını ve veri akışını gösterir. Yeşil ile işaretli katmanlar mevcut sistemde olmayan **yeni** bileşenlerdir.

ÖNERİLEN NEXT.JS WEB PLATFORM MİMARİSİ

Multi-tenant · Cloud-native · Semantic Search · Provider-abstracted AI



Şekil 10.1 — Önerilen Next.js + PostgreSQL + pgvector mimari diyagramı

10.2 Katman Sorumlulukları

- **Client (Next.js 16 + React + TypeScript + Tailwind + shadcn/ui):** 6 ana sayfa — dashboard, search, petitions, research/[id], chat, documents
- **API Gateway (Next.js API Routes):** 6 endpoint grubu — auth, legal-search, petitions, research, documents, admin. RBAC + rate limit + input validation
- **Python Legal Engine (FastAPI container):** 5 modül — engine-wrapper (modülerleştirilmiş engine.py), mcp-adapter (21→REST), quality-engine, petition-renderer (WeasyPrint+docx), source-connectors (async httpx)
- **AI Provider Abstraction:** 5 sağlayıcı — OpenAI, Gemini, Anthropic, OpenRouter, Ollama. Env-driven seçim, fallback, retry, cost-aware routing
- **Hybrid Search Engine:** 5 bileşen — Semantic (pgvector), Keyword (FTS+BM25), Metadata Filter, Cross-encoder Reranker, Query Expansion (LLM)
- **PostgreSQL 16 + pgvector:** 18+ tablo, partitioned legal_decisions (yıllara göre), HNSW index, JSONB metadata
- **Async Job Queue (Redis + BullMQ):** 4 worker tipi — Import Pipeline (9M+ belge), Source Sync (scheduled), Document Upload (user), AI Research (long-running)
- **Storage & Infra:** S3-compatible (MinIO), Redis cache, Vercel/Docker, Python container, observability (Sentry+OTel+Grafana)

10.3 İletişim Akışı

Tipik bir araştırma senaryosunda veri akışı: **Browser** → HTTPS → **Next.js API route** → (sync) **Python service** (HTTP) → (async) **Job Queue** → **PostgreSQL + pgvector**. SSE ile ilerleme tarayıcıya stream edilir. AI chat için: Browser → SSE → API route → Python service → AI provider (OpenAI/Gemini) → tool dispatch → source-connectors → sonuçlar AI'ya → AI cevabı SSE ile stream.

BÖLÜM 11

PostgreSQL + pgvector Mimari

Önerilen veritabanı şeması, 18+ tablodan oluşur ve mevcut SQLite'in 2 tabloluk yapısının yerine çok-kullanıcılı, audit-logged, versiyonlu, semantically-searchable bir sistem sunar. Aşağıda her tablonun DDL benzeri yapı verilmiştir.

11.1 Auth & Tenant Tabloları

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,          -- bcrypt/argon2
  full_name   TEXT,
  role        TEXT NOT NULL DEFAULT 'user', -- admin/lawyer/user/org_admin
  bar_no      TEXT,                    -- avukat sicil no
  email_verified BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT now(),
  last_login_at TIMESTAMPTZ,
  deleted_at  TIMESTAMPTZ             -- soft delete
);

CREATE TABLE organizations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  slug        TEXT UNIQUE NOT NULL,
  plan        TEXT DEFAULT 'free',    -- free/pro/enterprise
  seat_count  INT DEFAULT 1,
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE organization_members (
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  org_id      UUID REFERENCES organizations(id) ON DELETE CASCADE,
  role        TEXT NOT NULL,          -- org_admin/lawyer/paralegal
  PRIMARY KEY (user_id, org_id)
);

CREATE TABLE subscriptions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID REFERENCES organizations(id),
  stripe_cust_id TEXT,
  stripe_sub_id TEXT,
  plan        TEXT,
  status      TEXT,
  current_period_end TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE api_keys (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  key_hash    TEXT NOT NULL,
```

```

        name                TEXT,
        last_used_at        TIMESTAMPTZ,
        expires_at          TIMESTAMPTZ,
        created_at          TIMESTAMPTZ DEFAULT now()
    );

CREATE TABLE audit_logs (
    id                BIGSERIAL PRIMARY KEY,
    user_id           UUID REFERENCES users(id),
    org_id            UUID,
    action            TEXT NOT NULL,          -- create_petition, search, export, login
    entity_type       TEXT,                  -- petition, case, document
    entity_id         UUID,
    ip_address        INET,
    user_agent        TEXT,
    metadata          JSONB,
    created_at        TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_audit_user_time ON audit_logs(user_id, created_at DESC);
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);

```

11.2 Case & Document Tabloları

```

CREATE TABLE cases (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id            UUID REFERENCES organizations(id),
    title             TEXT NOT NULL,
    case_no           TEXT,                  -- mahkeme esas no
    court             TEXT,
    track             TEXT,                  -- iş/boşanma/idare/...
    status            TEXT DEFAULT 'open',   -- open/closed/archived
    created_by        UUID REFERENCES users(id),
    created_at        TIMESTAMPTZ DEFAULT now(),
    updated_at        TIMESTAMPTZ DEFAULT now(),
    deleted_at        TIMESTAMPTZ
);
CREATE INDEX idx_cases_org ON cases(org_id, created_at DESC);

CREATE TABLE case_parties (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id           UUID REFERENCES cases(id) ON DELETE CASCADE,
    role              TEXT NOT NULL,        -- plaintiff/defendant/witness
    name              TEXT,
    tckn              TEXT,
    address           TEXT,
    metadata          JSONB
);

CREATE TABLE documents (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id            UUID REFERENCES organizations(id),
    case_id           UUID REFERENCES cases(id),
    user_id           UUID REFERENCES users(id),
    filename          TEXT NOT NULL,
    mime_type         TEXT,
    size_bytes        BIGINT,

```

```

    s3_key          TEXT,                -- S3/MinIO object key
    parsed_text     TEXT,
    parse_status    TEXT,                -- pending/parsed/failed
    source          TEXT,                -- user_upload/court/scrape
    created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE document_chunks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id     UUID REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index     INT NOT NULL,
    chunk_text      TEXT NOT NULL,
    metadata        JSONB,              -- page, section, heading
    UNIQUE (document_id, chunk_index)
);

-- pgvector: 1536d OpenAI text-embedding-3-large
CREATE TABLE document_embeddings (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    chunk_id        UUID REFERENCES document_chunks(id) ON DELETE CASCADE,
    embedding        vector(1536) NOT NULL,
    model           TEXT DEFAULT 'text-embedding-3-large',
    created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_doc_emb_hnsw ON document_embeddings
    USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);

```

11.3 Legal Source Tabloları (9M+ karar)

```

CREATE TABLE legal_sources (
    id              SERIAL PRIMARY KEY,
    name            TEXT UNIQUE NOT NULL, -- yargitay/danistay/emsal/aym/resmi_gazete/mevzuat
    base_url        TEXT,
    api_url         TEXT,
    scraper_config   JSONB,              -- rate_limit, headers, parser
    last_synced_at  TIMESTAMPTZ,
    enabled         BOOLEAN DEFAULT TRUE
);

-- Partitioned by year for 9M+ decisions
CREATE TABLE legal_decisions (
    id              BIGSERIAL,
    source_id       INT REFERENCES legal_sources(id),
    source_doc_id   TEXT,                -- portal-side unique ID
    court           TEXT,
    court_chamber   TEXT,                -- 9. HD, 10. HD, vs.
    decision_number TEXT,                -- E.2024/123 K.2024/456
    case_number     TEXT,
    decision_date   DATE,
    document_type   TEXT,                -- decision/ruling/verdict
    title           TEXT,
    full_text       TEXT,
    summary         TEXT,
    metadata        JSONB,
    scraped_at      TIMESTAMPTZ DEFAULT now(),

```

```

        PRIMARY KEY (id, decision_date)
    ) PARTITION BY RANGE (decision_date);

CREATE TABLE legal_decisions_2024 PARTITION OF legal_decisions
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
-- ... diğer yıllar

CREATE TABLE legal_decision_chunks (
    id                BIGSERIAL PRIMARY KEY,
    decision_id       BIGINT,
    chunk_index       INT,
    chunk_text        TEXT,
    metadata          JSONB
) PARTITION BY RANGE (chunk_index);

CREATE TABLE legal_decision_embeddings (
    id                BIGSERIAL PRIMARY KEY,
    chunk_id          BIGINT REFERENCES legal_decision_chunks(id) ON DELETE CASCADE,
    embedding          vector(1536) NOT NULL,
    model             TEXT DEFAULT 'multilingual-e5-large',
    created_at        TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_legal_emb_hnsw ON legal_decision_embeddings
    USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);

CREATE TABLE statutes (
    id                SERIAL PRIMARY KEY,
    name              TEXT NOT NULL,           -- 4857 sayılı İş Kanunu
    statute_no        TEXT,                   -- 4857
    source_url        TEXT,
    current_version_id INT,
    created_at        TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE statute_articles (
    id                SERIAL PRIMARY KEY,
    statute_id        INT REFERENCES statutes(id),
    article_no        TEXT,                   -- 18, 20/A
    version           INT DEFAULT 1,          -- v1, v2, v3
    title             TEXT,
    body_text         TEXT,
    effective_from    DATE,
    effective_to      DATE,                   -- null = current
    source_url        TEXT,
    UNIQUE (statute_id, article_no, version)
);
CREATE INDEX idx_statute_current ON statute_articles(statute_id)
    WHERE effective_to IS NULL;

```

11.4 Petition & Research Tabloları

```

CREATE TABLE petitions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id            UUID REFERENCES organizations(id),
    case_id           UUID REFERENCES cases(id),
    user_id           UUID REFERENCES users(id),

```

```

    petition_type    TEXT,                                -- idare/bosanma/is/...
    current_version_id UUID,                              -- FK to petition_versions
    status           TEXT DEFAULT 'draft',                -- draft/reviewed/final/exported
    created_at       TIMESTAMPTZ DEFAULT now(),
    updated_at       TIMESTAMPTZ DEFAULT now(),
    deleted_at       TIMESTAMPTZ
);
CREATE INDEX idx_petitions_org_user ON petitions(org_id, user_id, created_at DESC);

CREATE TABLE petition_versions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    petition_id       UUID REFERENCES petitions(id) ON DELETE CASCADE,
    version_no        INT NOT NULL,
    body_text         TEXT NOT NULL,
    form_data         JSONB,                              -- structured user input
    quality_score     INT,
    quality_report    JSONB,
    ai_run_id         UUID,                              -- AI generation run reference
    created_by        UUID REFERENCES users(id),
    created_at        TIMESTAMPTZ DEFAULT now(),
    UNIQUE (petition_id, version_no)
);

CREATE TABLE petition_reviews (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    petition_version_id UUID REFERENCES petition_versions(id),
    review_type       TEXT,                              -- auto_regex/llm_judge/user
    score             INT,
    findings          JSONB,
    reviewed_by       UUID REFERENCES users(id),
    created_at        TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE research_sessions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id            UUID REFERENCES organizations(id),
    user_id           UUID REFERENCES users(id),
    case_id           UUID REFERENCES cases(id),
    petition_id       UUID REFERENCES petitions(id),
    query             TEXT,
    track             TEXT,
    status            TEXT DEFAULT 'running', -- running/completed/failed
    started_at        TIMESTAMPTZ DEFAULT now(),
    completed_at      TIMESTAMPTZ
);

CREATE TABLE research_traces (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id        UUID REFERENCES research_sessions(id) ON DELETE CASCADE,
    step_order        INT,
    step_type         TEXT,                              -- intent_detect/query_gen/search/verify
    step_name         TEXT,
    input_data        JSONB,
    output_data       JSONB,
    duration_ms       INT,
    started_at        TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ
);

```

```

);

CREATE TABLE citations (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  petition_version_id UUID REFERENCES petition_versions(id),
  source_type       TEXT,                -- yargitay/danistay/aym/statute
  source_id         TEXT,
  citation_text     TEXT,                -- "Yargıtay 9. HD. E.2024/123 K.2024/456
"
  page_ref         TEXT,
  verified         BOOLEAN DEFAULT FALSE,
  metadata         JSONB
);

CREATE TABLE ai_runs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID REFERENCES users(id),
  session_id       UUID REFERENCES research_sessions(id),
  provider         TEXT,                -- openai/gemini/anthropic/ollama
  model            TEXT,                -- gpt-4o, claude-3-5-sonnet, qwen-14b
  input_tokens     INT,
  output_tokens    INT,
  cost_usd         DECIMAL(10,4),
  duration_ms      INT,
  status           TEXT,
  error_message    TEXT,
  created_at       TIMESTAMPTZ DEFAULT now()
);

```

11.5 pgvector Index Stratejisi

HNSW (Hierarchical Navigable Small World) index tercih edilmiştir. Parametreler: **m=16** (her node'un bağlantı sayısı), **ef_construction=64** (inşaat sırası arama genişliği). Bu parametreler 1M-10M vektör için optimal kabul edilir. Sorgu zamanında **ef_search** (varsayılan 40) ayarlanabilir — daha yüksek değer daha doğru sonuç ama daha yavaş sorgu. Cosine similarity (*vector_cosine_ops*) kullanılır çünkü e5/OpenAI embedding'leri normalize edilmiştir.

BÖLÜM 12

API Taslağı

Önerilen REST API, 6 endpoint grubundan ve toplam ~50 endpoint'ten oluşur. Tüm endpoint'ler **JWT auth** gerektirir (health ve login hariç), **RBAC** ile korunur, **rate limit**'e tabidir. Input validation Zod şemaları ile yapılır. AI streaming endpoint'leri SSE kullanır.

12.1 Auth Endpoints

Method	Endpoint	İşlev	Auth
POST	/api/auth/register	Email + password ile kayıt. Email verification gönderir.	Public
POST	/api/auth/login	Login → JWT access + refresh token	Public
POST	/api/auth/refresh	Refresh token → yeni access token	Refresh
POST	/api/auth/logout	Token blacklist'e ekle	JWT
POST	/api/auth/verify-email	Email verification token	Public
POST	/api/auth/reset-password	Şifre sıfırlama talebi + yeni şifre	Public
GET	/api/auth/me	Mevcut kullanıcı bilgisi	JWT
GET	/api/auth/oauth/[provider]	Google/GitHub OAuth (opsiyonel)	Public

Tablo 12.1 — Auth endpoints

12.2 Legal Search Endpoints

Method	Endpoint	İşlev	Rate
POST	/api/legal/search/bedesten	Yargıtay+Danıştay+Emsal araması (sync, portal)	10/dk
POST	/api/legal/search/deep	Sayfa sayfa derin arama (async job başlatır)	5/dk
POST	/api/legal/search/mevzuat	Mevzuat arama (versiyonlu)	20/dk
POST	/api/legal/search/aym	AYM kararı arama (BB/ND)	20/dk
POST	/api/legal/search/resmi-gazete	Resmî Gazete arama (tarih aralığı)	20/dk
POST	/api/legal/search/semantic	Semantic search (pgvector + rerank)	30/dk
POST	/api/legal/search/hybrid	Hybrid: semantic + keyword + metadata + rerank	30/dk

Method	Endpoint	İşlev	Rate
GET	/api/legal/document/[source]/[id]	Belge detayı (Yargıtay/Danıştay/AYM)	60/dk
POST	/api/legal/expand-query	AI query expansion (sinonim/kavram)	10/dk

Tablo 12.2 — Legal search endpoints

12.3 Petition & Research Endpoints

Method	Endpoint	İşlev
POST	/api/petitions	Yeni dilekçe oluştur (form data ile)
GET	/api/petitions	Kullanıcı/org dilekçelerini listele (pagination)
GET	/api/petitions/[id]	Dilekçe detayı (current version)
POST	/api/petitions/[id]/generate	AI ile dilekçe metni üret (SSE stream)
POST	/api/petitions/[id]/review	17-boyut kalite kontrol çalıştır
POST	/api/petitions/[id]/repair	AI ile düşük skorlu dilekçeyi düzelt
GET	/api/petitions/[id]/versions	Versiyon listesi (v1, v2, v3...)
GET	/api/petitions/[id]/versions/[v]	Belirli versiyon metni
POST	/api/petitions/[id]/versions/compare	İki versiyon diff (unified diff format)
POST	/api/petitions/[id]/export-pdf	PDF üret (WeasyPrint), S3'e yükle
POST	/api/petitions/[id]/export-docx	DOCX üret (python-docx)
POST	/api/research/sessions	Yeni AI araştırma oturumu başlat
GET	/api/research/sessions/[id]	Oturum durumu + trace adımları
GET	/api/research/sessions/[id]/stream	SSE: gerçek zamanlı trace adımları

Tablo 12.3 — Petition & Research endpoints

12.4 Document, Chat & Admin Endpoints

- **POST** /api/documents/upload — PDF/DOCX/TXT yükle, async parse job'ı başlat
- **GET** /api/documents — Kullanıcı belgeleri listesi
- **POST** /api/documents/[id]/search — Belge içinde semantic search
- **DELETE** /api/documents/[id] — Belge + embedding sil

- **POST** /api/chat/sessions — Yeni chat oturumu
- **POST** /api/chat/sessions/[id]/messages — Mesaj gönder (SSE stream)
- **GET** /api/chat/sessions/[id]/messages — Mesaj geçmişi
- **GET** /api/admin/users — Kullanıcı listesi (admin only)
- **GET** /api/admin/audit-logs — Audit log sorgu (admin only)
- **GET** /api/admin/ai-runs — AI kullanım istatistikleri (admin only)
- **GET** /api/admin/sources — Hukuk kaynağı sync durumu (admin only)
- **GET** /api/health — Health check (public, /api/health/simple)

BÖLÜM 13

Migration Planı (19 Faz)

Migration planı, 19 fazdan oluşur ve toplam **5-7 ay** sürmesi öngörülür. Faz 1 (Kod Analizi) bu raporla tamamlanmıştır. Her faz, belirli çıktılara sahiptir ve bir sonraki fazın başlaması için bağımlılık teşkil eder. Kritik yol (critical path): Faz 2 → Faz 3 → Faz 4 → Faz 5 → Faz 7 → Faz 8 → Faz 10 → Faz 11.

Faz	Adı	Süre	Çıktı / Teslimat	Bağımlılık
1	Kod Analizi & Mimari Doküman	2 hf	Bu rapor (TAMAMLANDI)	—
2	Database Şema (PostgreSQL + pgvector)	2 hf	Prisma schema, 18+ tablo, migration SQL	F1
3	Legal Source Abstraction (Python)	2 hf	Async httpx source-connectors modülü	F1
4	Python Engine API Service (FastAPI)	3 hf	engine-wrapper, mcp-adapter (18/21 araç), Dockerfile	F2, F3
5	Next.js Application Scaffold	2 hf	Next.js 16, TypeScript, Tailwind, shadcn/ui, Prisma	F2
6	Authentication (NextAuth + JWT + RBAC)	2 hf	Login/register/verify/reset, 4 rol	F5
7	Legal Search (Mevcut araçların REST'e çevrimi)	2 hf	5 search endpoint, Redis rate-limit	F4, F6
8	Semantic Search (pgvector)	3 hf	Embedding pipeline, HNSW index, semantic endpoint	F2, F7
9	Hybrid Search + Reranking	2 hf	BM25+vector+rerank orchestrator	F8
10	AI Orchestration (Provider Abstraction)	3 hf	5 provider, env-driven router, fallback	F4
11	Dilekçe Sistemi	3 hf	Generate/review/repair, 12 şablon, AI stream	F7, F10
12	Research Trace UI	2 hf	Şeffaf AI adım gösterimi, SSE stream	F11
13	Document Upload + Parse + Embed	2 hf	PDF/DOCX/TXT, OCR, async job, user index	F8
14	Versioning (Petition + Statute)	2 hf	Versiyon geçmişi, diff, rollback, mevzuat v1/v2/current	F11
15	PDF/DOCX Export	1 hf	WeasyPrint + python-docx, S3 storage	F11
16	Admin Panel	2 hf	User mgmt, audit logs, usage, source sync	F6, F7
17	Performance Optimization	2 hf	Cache, query plan, index tuning, load test	F9, F11
18	Security Audit	2 hf	CSRF/XSS/SQLi/SSRF, prompt injection, rate limit	F16

Faz	Adı	Süre	Çıktı / Teslimat	Bağımlılık
19	Production Deployment	1 hf	CI/CD, monitoring, runbook, backup strategy	F17, F18

Tablo 13.1 — 19 fazlık migration yol haritası

13.1 Kritik Yol (Critical Path)

F1 (Kod Analizi) → F2 (DB) → F4 (Python Service) → F5 (Next.js) → F7 (Legal Search) → F8 (Semantic) → F9 (Hybrid) → F10 (AI) → F11 (Dilekçe) → F12 (Research Trace) → F15 (PDF/DOCX) → F19 (Deploy). Bu kritik yol yaklaşık 27 hafta (~6.5 ay) sürer. Paralel yapılabilecek fazlar: F3 || F5 || F6, F8 || F10, F12 || F13 || F14 || F16.

BÖLÜM 14

Tahmini Modül Listesi

Önerilen web platformu üç ana katmanda organize edilir: **Frontend (Next.js)**, **Backend (Next.js API + Python Service)**, **Shared (Types & Schemas)**. Aşağıda her katmanın modül listesi ve dosya yapısı özeti verilmiştir.

14.1 Frontend Modülleri (app/)

```

app/
├── (auth)/
│   ├── login/page.tsx
│   ├── register/page.tsx
│   ├── verify-email/page.tsx
│   └── reset-password/page.tsx
├── (dashboard)/
│   ├── layout.tsx                # Sidebar + topbar
│   ├── page.tsx                  # /dashboard ana sayfa
│   ├── search/
│   │   ├── page.tsx              # /search
│   │   └── [resultId]/page.tsx    # /search/[id]
│   ├── petitions/
│   │   ├── page.tsx              # Liste
│   │   ├── new/page.tsx          # Yeni dilekçe
│   │   └── [id]/
│   │       ├── page.tsx          # Detay/editör
│   │       ├── versions/page.tsx # Versiyon geçmişi
│   │       └── compare/page.tsx   # Diff
│   ├── research/
│   │   ├── page.tsx              # Oturum listesi
│   │   └── [id]/page.tsx         # Trace görselleştirme
│   ├── chat/
│   │   ├── page.tsx              # Chat listesi
│   │   └── [id]/page.tsx         # Chat ekranı (SSE)
│   ├── documents/
│   │   ├── page.tsx              # Liste
│   │   └── [id]/page.tsx         # Detay + search
│   └── settings/
│       ├── profile/page.tsx
│       ├── organization/page.tsx
│       └── api-keys/page.tsx
├── admin/
│   ├── users/page.tsx
│   ├── audit-logs/page.tsx
│   ├── sources/page.tsx
│   └── ai-usage/page.tsx
└── api/                          # API routes (Bölüm 12)

```

14.2 Backend Modülleri

Next.js API katmanı 6 modülden oluşur: *auth* (NextAuth config + JWT), *cases* (case CRUD), *legal-search* (5 search endpoint), *petitions* (generate/review/versions/export), *research* (sessions/traces), *documents*

(upload/parse/search), *chat* (sessions/messages), *admin* (users/audit/sources), *ai-provider* (provider abstraction), *embeddings* (queue trigger), *queue* (BullMQ config).

Python servis modülleri 5 ana modülden oluşur: (1) *engine-wrapper* — modülerleştirilmiş *engine.py* (*engine/main.py*, *engine/llm.py*, *engine/research.py*, *engine/petition.py*, *engine/cleaner.py*, *engine/citation.py*); (2) *mcp-adapter* — 21 MCP aracının FastAPI router'lara çevrilmesi; (3) *quality-engine* — *quality.py* + *motor/pipeline.py* (sync→threadpool); (4) *petition-renderer* — WeasyPrint + python-docx + Jinja2; (5) *source-connectors* — async httpx + Redis rate-limiter.

14.3 Shared Modüller

- **types/** — TypeScript tip tanımları (User, Case, Petition, Document, LegalDecision, Citation, vb.)
- **schemas/** — Zod input validation şemaları (her API endpoint için request/response)
- **db/** — Prisma schema + client + migration scripts
- **lib/auth/** — JWT utilities, RBAC helpers, session management
- **lib/ai/** — Provider abstraction interface, OpenAI/Gemini/Anthropic/OpenRouter/Ollama implementations
- **lib/search/** — Hybrid search orchestrator, BM25, reranker wrapper
- **lib/embeddings/** — Embedding pipeline, chunking utilities
- **lib/queue/** — BullMQ job definitions (import, source-sync, doc-upload, ai-research)

BÖLÜM 15

İlk Sprint Görevleri

Sprint 1, migration'ın temelini atan kritik bir sprint'tir. **2-3 hafta** sürmesi öngörülür ve 8 ana görevden oluşur. Bu sprint'in başarısı, sonraki tüm fazların sağlam bir temel üzerinde inşa edilmesini sağlar.

#	Görev	Süre	Çıktı	Sorumlu
S1.1	Next.js proje scaffold + Prisma + PostgreSQL kurulumu	2 gün	Çalışan Next.js 16 + DB connection	Frontend
S1.2	DB şema migration (18 tablo, Prisma)	3 gün	schema.prisma + migration SQL	Backend
S1.3	Python engine containerization	3 gün	Dockerfile + docker-compose + FastAPI skeleton	Python
S1.4	Auth temel (email/password + JWT)	3 gün	register/login/refresh + RBAC middleware	Backend
S1.5	Mevcut MCP araçlarının REST'e çevrilmesi (18/21)	4 gün	5 search endpoint + document endpoint	Python+Backend
S1.6	Temel dashboard UI	3 gün	/dashboard sayfası + sidebar + topbar	Frontend
S1.7	Legal search UI mockup	2 gün	/search sayfası (mock data)	Frontend
S1.8	PDF export (WeasyPrint) iskelet	2 gün	/api/petitions/[id]/export-pdf	Python

Tablo 15.1 — Sprint 1 görevleri (2-3 hafta)

15.1 Sprint 2 Önizleme

Sprint 2 (3-4 hafta) odak alanları: (a) Semantic search altyapısı (pgvector + multilingual-e5-large + HNSW index); (b) 9M+ karar import pipeline'ı (async job queue, batch processing, increment indexing); (c) AI provider abstraction (OpenAI/Gemini/Anthropic); (d) Petition generate pipeline (engine.py modülerleştirilmiş hali); (e) Research trace SSE streaming.

15.2 Sprint 3 Önizleme

Sprint 3 (3-4 hafta) odak alanları: (a) Hybrid search (BM25+vector+rerank); (b) Query expansion (LLM tabanlı); (c) Document upload + parse + embed; (d) Versiyonlama (petition + statute); (e) DOCX export; (f) Admin panel iskelet; (g) Performance optimization ilk tur.

Sonuç ve Sonraki Adımlar

BetterSaul projesi, Türkiye hukuk kaynakları üzerinde çalışan olgun bir regex-tabanlı hukuk AI motorudur. Mevcut sistem **mühendislik açısından değerli** (17-boyut kalite kontrol, halüsinasyon kontrolü, 21 MCP aracı, 6 hukuk kaynağı entegrasyonu) ancak **mimari açısından web ölçeğine uygun değil** (tek-kullanıcılı, semantic search yok, desktop-only bağımlılıklar, Qwen-only AI).

Migration'ın **4 kritik dönüşümü**: (1) Semantic search eklenmesi — pgvector + multilingual-e5 + cross-encoder rerank ile; (2) Python motorunun servisleştirilmesi — FastAPI container, mevcut 21 MCP aracının 18'i doğrudan REST'e çevrilir; (3) Çok-kullanıcılı mimari — PostgreSQL 18+ tablo, RBAC, multi-tenant; (4) AI provider abstraction — OpenAI/Gemini/Anthropic/OpenRouter/Ollama tek arayüz altında. Toplam migration süresi **5-7 ay** (19 faz).

Hemen Başlanacak İlk 3 Adım

- **1. Sprint 1 başlat**: Next.js scaffold + PostgreSQL + Prisma + Python FastAPI container. Bu rapordaki 18-tablo şemasını Prisma'ya çevir, Docker Compose ile local dev ortamı kur.
- **2. engine.py modülerleştir**: 7.857 satırlık tek dosyayı 6 modüle böl (main/llm/research/petition/cleaner/citation). Bu, sonraki tüm Python servis çalışmasının temelidir.
- **3. Semantic search iskelet**: pgvector extension kur, multilingual-e5-large embedding pipeline yaz, ilk 100K kararı indexle. Bu, kullanıcıların ilk göreceği "vay be" deneyimini yaratacak özelliktir.

Korunması Gereken Değerler (Hatırlatma)

- **quality.py + motor/pipeline.py** — 17-boyut regex kalite kontrolü (saf Python, doğrudan taşınabilir)
- **Halüsinasyon kontrolü** — TCKN/tanık/tutar/prompt sızıntısı tespiti (kaynak-karşılaştırma)
- **21 MCP aracı** — 18'i doğrudan REST'e çevrilebilir, 3'ü adaptasyon gerektirir
- **10 dava türü tespiti** — legal_tracks.detect_track() regex sınıflandırıcı
- **12 dilekçe şablonu** — templates.py KINDS sözlüğü + petition_packs.py
- **6 hukuk kaynağı entegrasyonu** — Yargıtay/Danıştay/Emsal/AYM/RG/Mevzuat scraper
- **Prompt disiplini** — system_prompt.txt + petition_prompt.txt (112 satır A-M bölümlü)

Hedef ürün: "BetterSaul'un web sitesine çevrilmiş hali" değil; **Türkiye hukuk kaynakları üzerinde çalışan AI destekli Legal Research + Legal Intelligence Platform**. Modern, özgün, hızlı, kaynak gösteren, semantic search destekli, AI destekli, hukuk araştırmasına odaklı, dilekçe oluşturabilen, milyonlarca belgeyle ölçeklenebilen, güvenli, production-ready.